Sadaf Zehra

sadafhussain636@gmail.com | +92 331 2724602 | Karachi, Pakistan
Website | LinkedIn | GitHub | ORCID: 0009-0009-7530-9006

Education

- **MS in Data Science** — Karachi, Pakistan
  NED University of Engineering and Technology — *Aug 2024 – Jun 2026*
  - CGPA: **3.81/4.00** | Merit-based scholarship for three semesters
  - Thesis: *Few-Shot Font Generation Using Conditional Variational Autoencoder*
  - Supervisor: Dr. Muhammad Umer Farooq
- **BS in Software Engineering** — Karachi, Pakistan
  University of Karachi — *Jan 2018 – Dec 2021*
  - CGPA: 3.53/4.00
  - Final Year Project: *LungCare*, a chest X-ray lung damage assessment system built with real hospital data

Publications

[1] M. Khanum, H. Ali, **S. Zehra**, S. Rajput, and Q. Mahmood, "Performance Analysis of Machine Learning Algorithms for Air Quality Index (AQI) Estimation," in *Proc. International Conference of Frontiers of Engineering and Emerging Technologies (FET'26)*, IEEE, 2026. DOI: 10.1109/FET68771.2026.11601421

- Predicted the Air Quality Index from six pollutants (PM2.5, PM10, NO, SO_2, CO, O_3) with Decision Tree, Random Forest and XGBoost, using forward feature selection on OpenAQ data from four cities in the USA, Austria, Spain and the UK (2019–2024). **Presented the paper** at the conference.

Experience

- **FAST – National University of Computer and Emerging Sciences (FAST-NUCES)** — Jan 2025 – Present
  Instructor, Computer Science — *Karachi, Pakistan*
  - Teach labs for Artificial Intelligence, Database Systems, Computer Networks, Object-Oriented Programming and Computer Organization & Assembly Language (COAL); write the lab manuals and design all lab tasks.
  - Supervise and evaluate student course projects each semester; serve as a jury member for Final Year Project (FYP) evaluations.
  - Supervised a team of four students who built *Baymax*, a deployed multi-agent AI career assistant (LangChain, LangGraph, CrewAI) with RAG-based resume review (ChromaDB) and a live job-search API.
- **Bahria University** — Sep 2026 – Present
  Visiting Faculty, Computer Science — *Karachi, Pakistan*
  - Teach Machine Learning and Data Science labs; write lab manuals and tasks; supervise and evaluate student course projects.
- **Aawaz Institute of Media & Management Sciences (AIIMS)** — Jan 2024 – Jun 2024
  Visiting Faculty, Computer Science — *Karachi, Pakistan*
  - Taught Compiler Construction and Data Structures.
- **University of Karachi** — Jul 2023 – Dec 2023
  Visiting Faculty, Computer Science — *Karachi, Pakistan*
  - Taught Introduction to Computer Science.
- **Faralone Enterprises and Consultant** — Apr 2022 – Nov 2023
  ServiceNow Intern, then ServiceNow Developer (from Jun 2022) — *Karachi, Pakistan*
  - Worked on the ServiceNow platform and earned the ServiceNow Certified System Administrator certification (Jun 2022).
- **The Disrupt Labs** — Oct 2021 – Mar 2022
  Computer Vision Intern — *Karachi, Pakistan*
  - Built real-time computer vision models (about 15 ms per image on GPU) for industrial use on factory camera images.
  - Worked on key projects: **Smoke Detection** (EfficientNetB2, YOLOv5), **Package Detection** (YOLOv4-tiny), **Vehicle Classification** and **QR Code Recognition** (see Professional Projects).

Professional Projects

- **Industrial Smoke Detection: From Image Classification to Real-Time Localization** — Jan 2022 – Mar 2022
  Tools: Python, PyTorch, YOLOv5, TensorFlow/Keras, EfficientNetB2, Roboflow, Data Annotation [Code]
  - Built the dataset from factory camera and Kaggle images and **manually annotated about 1,000 images with bounding boxes in Roboflow**; first fine-tuned an ImageNet-pretrained EfficientNetB2 smoke / no-smoke classifier with data augmentation.

- Then trained a YOLOv5s detector to locate the smoke (3,816 train / 859 validation images), reaching **94.7% mAP@0.5** at about **15 ms per image** on a Tesla T4 GPU.

- **Real-Time Package Detection on an Industrial Line with YOLOv4-Tiny** Nov 2021 – Jan 2022
  Tools: Python, Darknet, YOLOv4-tiny, CUDA/cuDNN, Roboflow [Code]
  - Fine-tuned a pretrained YOLOv4-tiny detector on a custom dataset of factory camera images with a full GPU training pipeline in Darknet.
  - Reached **95.72% mAP@0.50** (precision 1.00, recall 0.85, F1 0.92) with **zero false positives** at about **15.7 ms per image**, fast enough for real-time monitoring.

Academic Projects

- **MS Thesis: Few-Shot Font Generation Using Conditional Variational Autoencoder** Aug 2025 – Jun 2026
  Tools: Python, TensorFlow/Keras, Conditional VAE, Few-Shot Learning, SSIM, FID [Code]
  - Built a conditional VAE that writes characters in a new handwriting style after seeing only **five example images**, trained with **5-way 5-shot** episodes on Omniglot so it works on alphabets never seen in training (BCE + MAE + SSIM loss, KL annealing).
  - Achieved **SSIM 0.782** and **FID 12.21**, compared with published results of eight few-shot font generation methods (FUNIT, MX-Font, DG-Font, CF-Font, NTF-Loc, VQ-Font, Diff-Font, SGD-Font).

- **GLANCE: Gaze-Driven Learner Profiling for Adaptive Pragmatic Code Explanations** 2026
  Tools: Python, PyTorch, Hugging Face Transformers, Qwen2.5-Coder, EMIP Eye-Tracking Data, scikit-learn, RSA [Code]
  - Aligned eye-tracking fixations of **154 programmers** reading Java code (EMIP dataset) to code tokens and tested whether **code-LLM surprisal** (Qwen2.5-Coder 0.5B / 1.5B) predicts gaze duration for different reader groups, using held-out log-likelihood per participant.
  - Profiled **unseen developers from 1, 3 or 5 short gaze windows** (N-way few-shot identification and class prediction with prototypes and a learned NCA metric), and explained each decision with coefficients, permutation importance and decision rules.
  - Fed the profile into a **Rational Speech Acts** speaker that chooses among LLM code explanations by trading informativity against a gaze-fitted reading cost, with retrieval grounding and checks for hallucinated identifiers.

- **STALE: Stale-Value Tracing in Artificial and Learner Environments** 2026
  Tools: Python, PyTorch, Hugging Face Transformers, Qwen2.5-Coder, Linear Probes, Activation Patching, ACT-R [Code]
  - Built a benchmark of **400 generated Python programs** that vary the number of variables (2–5), re-assignments of the queried variable (0–4) and pattern (augmented, swap, loop), with execution-verified answers and error labels (stale value, other variable).
  - Tested code LLMs with 0, 2 and 4 demonstrations (exact McNemar tests) and located where variable values are stored with **layer-wise probes and activation patching**.
  - Modelled tracing errors with an **ACT-R style working-memory model**, estimated a learner's capacity from a few answers with a Bayesian posterior, and compared personalised and fixed hint policies in simulation.

- **TAILOR: Taste-Aware Interpretable Learning of Reader Preferences** 2026
  Tools: Python, PyTorch, Hugging Face Transformers, Qwen2.5-Coder, scikit-learn, Mixture of Experts, EM [Code]
  - Generated **192 code explanations** (24 Python functions in a 2×2×2 design of detail, example and confidence) and described each with nine interpretable features, including readability as reader-LM surprisal.
  - Trained a **sparse mixture-of-experts reward model** (L1 Bradley–Terry experts, fitted with EM) on simulated developer preferences, and measured recovery of hidden developer types, **few-shot adaptation from 1–16 comparisons**, and the faithfulness of contribution-contrast versus routing-weight explanations.

- **Beyond English Identifiers: Modelling the Second-Language Cost of Program Comprehension** 2026
  Tools: Python, PyTorch, Hugging Face Transformers, Qwen2.5, SmolLM2, scikit-learn, Bayesian Inference [Code]
  - Renamed the identifiers of 20 Python functions in four styles (English, abbreviated, **Roman Urdu**, neutral), verified all 80 variants by execution, and scored identifier-word familiarity with an English-centred and a multilingual LM ($r = 0.87$).
  - Generated code explanations in English, Roman Urdu and Urdu script and checked them with back-translation: the model **wrote Roman Urdu in only 1 of 16 cases**, a gap for a language variety that millions of programming students use.
  - Validated a comprehension model and a few-shot explanation-language policy on simulated classroom data; a vocabulary prior plus a few quiz answers matched the best language for **86% of students**.

- **LangFont: Language-Guided Few-Shot Font Generation with Diffusion** 2026
  Tools: Python, PyTorch, Stable Diffusion 1.5, LoRA (PEFT), CLIP, OpenCV, FID/KID [Code]
  - Built a font generator that takes 1–10 example letters plus a text such as "bold, rounded, handwritten" and draws any missing character: Stable Diffusion 1.5 fine-tuned with LoRA, plus a style encoder (CLIP + small CNN) that turns the examples into extra tokens next to the text.
  - Built a dataset of **300 Google Fonts** (62 characters each) with style descriptions generated by OpenCV shape analysis; tested on 38 fonts from 30 unseen font families.

- **Cut FID from 184.6 to 54.1** (KID×1000 from 94.3 to 7.8) against a thesis-style CVAE baseline with 5 references, with higher CLIP image similarity (0.877 vs 0.776); the CVAE kept a higher SSIM (0.818 vs 0.734).
- Ablations showed the reference images carry most of the style (text-only FID 73.7 vs 54.1) and quality stays stable from 1 to 10 references.

◦ **Seeing with Words: Language-Guided Prototypes for One-Shot Recognition** 2026
Tools: Python, PyTorch, OpenCLIP (ViT-B/32), Prototypical Networks, scikit-learn, CUB-200, mini-ImageNet [Code]

- Combined CLIP image prototypes with class text descriptions (CUB attributes, WordNet definitions) and compared eleven methods, including zero-shot text, image–text fusion, linear probes and a learned shot-aware gate, on 600 fixed 5-way episodes with 95% confidence intervals.
- Adding text raised **1-shot accuracy from 81.7% to 96.1% on CUB-200 (+14.3 points)** and from 88.3% to 99.0% on mini-ImageNet (+10.7); the gain shrank to about +2 points at 5-shot. All gains were significant on paired episodes.
- The learned gate shifted weight to text when examples were scarce (83% at 1-shot vs 60% at 5-shot on CUB), but simple validation-tuned fusion stayed best, and long descriptions gave no gain over class names.

◦ **When Examples Mislead: Few-Shot Prompting and Object Hallucination in Small VLMs** 2026
Tools: Python, PyTorch, Hugging Face Transformers, Qwen2-VL-2B, SmolVLM, 4-bit Quantization, POPE [Code]

- Tested whether 0, 1, 3 or 5 in-context examples reduce object hallucination in two small vision-language models (Qwen2-VL-2B, SmolVLM) on 900 balanced POPE questions, scoring yes/no token probabilities and reporting F1, hallucination rate, AUROC, bootstrap 95% CIs and McNemar tests.
- **Found that examples made the models worse**: for Qwen2-VL-2B, F1 fell from 0.832 to 0.761 and the **hallucination rate rose from 4.0% to 26.2%** at 5-shot (significant on the popular and adversarial splits), while SmolVLM became over-cautious (recall 0.64 to 0.53).

◦ **Systematic Review: Explainable Machine Learning for Distributed Big Data Systems** Jan – Jun 2026
Tools: PRISMA, Systematic Literature Review, Explainable AI (SHAP, LIME), LaTeX

- Reviewed **32 peer-reviewed studies** (2020–2025) on explainable ML in distributed big-data systems using the PRISMA method and six quality criteria (QA1–QA6), and built a taxonomy linking ML models, XAI techniques and platforms (Spark, Hadoop, Kubernetes, federated learning).
- Found that SHAP was the most used XAI method (19 of 32 studies) while only **6 of 32 studies** measured explanation quality (fidelity, stability, coherence), a key research gap.

◦ **Beyond the Mean Prototype: Multi-Modal Class Representations for Few-Shot Learning** 2026
Tools: Python, PyTorch, Prototypical Networks, Soft k-Means, Meta-Learning, Omniglot [Code]

- Improved Prototypical Networks by giving each class several prototypes (soft k-means) instead of one average, so classes with more than one style are described better; tested on 600 tasks per setting (5/20-way, 1/5-shot) on normal and mixed-style Omniglot.
- **Beat Prototypical Networks in all four 5-shot settings**, with every gain confirmed by 95% confidence intervals (best: 92.24% vs 91.77% on mixed-style 20-way 5-shot), and matched it in 1-shot as expected.

◦ **Safe Over Short: Learning to Trace Unseen Characters with Reinforcement Learning** 2026
Tools: Python, PyTorch, NumPy, Dynamic Programming, Q-Learning, DQN, PPO, Reward Shaping [Code]

- Turned handwriting tracing on Omniglot into an RL task where the pen slips 5% of the time (1,852 training, 500 unseen-alphabet test tasks) and implemented **ten RL methods from scratch**: dynamic programming, two Monte Carlo methods, Sarsa, Q-learning, Dyna-Q, DQN, REINFORCE, A2C and PPO.
- Showed that planning for slips beats the shortest path on **unseen alphabets (95.0% vs 75.4% success)**; Sarsa, Q-learning and Dyna-Q reached about **93% success**, close to the optimal 94.1%.
- Found deep RL agents cheating the reward (ending episodes early) and fixed it with potential-based reward shaping.

◦ **SCACM: Semantic Clustered Attention Conditioning for Few-Shot Generation on Unseen Alphabets** 2026
Tools: Python, PyTorch, Conditional VAE, Attention, IWAE, Omniglot [Code]

- Built a controlled test on unseen Omniglot alphabets comparing three ways to condition a few-shot conditional VAE (averaging, attention, clustered attention); clustered attention matched but did not beat averaging (IWAE NLL 105.68 vs 105.58 nats), traced to low variation inside Omniglot classes.

◦ **Not Every Pixel Votes Equally: Clustered Attention Pooling for Limited-Data Recognition** 2026
Tools: Python, PyTorch, ResNet, CIFAR-100, Grad-CAM, Calibration (ECE) [Code]

- Replaced global average pooling in a ResNet with (clustered) attention pooling on CIFAR-100; with 10% data attention raised accuracy (**40.47% vs 39.16%**) but hurt calibration (ECE 3.45% to 7.67%), and with full data all were about 74%. Analysed with Grad-CAM.

◦ **Multimodal Fake News Representation Learning with a Variational Autoencoder** 2026
Tools: Python, TensorFlow/Keras, Sentence-BERT, MobileNetV2, scikit-learn, XGBoost, t-SNE [Code]

- Fused Sentence-BERT text and MobileNetV2 image features for 22,392 FakeNewsNet articles (PolitiFact, GossipCop), trained a VAE to compress them into a 32-d latent space, explored it with PCA, t-SNE and k-means, and compared four classifiers (LR, RF, SVM, XGBoost).

- **Controllable Text-to-Image Generation with Stable Diffusion and CLIP-Based Evaluation** 2025
  Tools: Python, PyTorch, Hugging Face Diffusers, Stable Diffusion, CLIP [Code]
  * Built a Stable Diffusion pipeline and measured prompt fidelity with CLIP score across sampling steps, guidance, image size and negative prompts: the score levelled off after 25 steps, peaked at guidance 7.5, and dropped for portrait images (28.62 to 25.66) and negative prompts (28.51 to 27.26).
- **LungCare: Chest X-Ray Lung Damage Assessment System (BS Final Year Project)** 2021
  Tools: Python, OpenCV, Flask, React, Firebase [Code]
  * Built a web app (React, Flask, Firebase) that takes a chest X-ray and reports how badly the lungs are damaged, using real X-rays of COVID-19, tuberculosis and pneumonia patients from a hospital.
  * Designed the image analysis (lung-area masking, pixel-intensity analysis) with a medical doctor, who validated the results.

Skills

- **Programming Languages:** Python, SQL, JavaScript (React)
- **Deep Learning & Generative Modeling:** PyTorch, TensorFlow/Keras, VAEs, Diffusion Models (Stable Diffusion, LoRA fine-tuning, Hugging Face Diffusers), CLIP/OpenCLIP, Vision-Language Models (Qwen2-VL, SmolVLM), Few-Shot and Meta-Learning, Reinforcement Learning
- **LLMs & Computational Cognitive Modeling:** Hugging Face Transformers, Code LLMs (Qwen2.5-Coder), Surprisal-Based Reading Models, Rational Speech Acts, ACT-R Memory Modeling, Probing and Activation Patching, Mixture-of-Experts Reward Models, Eye-Tracking Data Analysis
- **Computer Vision:** OpenCV, YOLOv4/YOLOv5, Darknet, EfficientNet, ResNet, Grad-CAM, Roboflow
- **Data Science & Machine Learning:** scikit-learn, XGBoost, NumPy, Pandas, Matplotlib, Seaborn
- **Web, Cloud & Tools:** Flask, Firebase, Google Colab, CUDA, Git/GitHub, LaTeX, ServiceNow
- **Specialized Area:** Few-Shot Learning, Deep Generative Modeling, Vision-Language Models, Multimodal Learning, Computer Vision, Reinforcement Learning, LLMs for Program Comprehension
- **Research Skills:** Experimental Design, Ablation Studies, Confidence Intervals, Bootstrap and McNemar Tests, Generative Model Evaluation (FID, KID, SSIM, IWAE, CLIP score), Model Calibration (ECE), Systematic Literature Review (PRISMA), Academic Writing

Volunteer Experience

- **Tutor** Mar 2022 – Dec 2024
  Home-based coaching *Karachi, Pakistan*
  * Coached first- and second-year intermediate students, explaining difficult topics step by step and helping them build confidence.
  * Built teaching, mentoring and communication skills that I now use every day as a university instructor.

Online Certifications

- Certified Artificial Intelligence Developer, Presidential Initiative for Artificial Intelligence and Computing (PIAIC), Jan 2023
- ServiceNow Certified System Administrator, ServiceNow, Jun 2022
- **Online courses completed:** Reinforcement Learning, RWTH Aachen University (edX); Stanford CS236: Deep Generative Models; Stanford CS330: Deep Multi-Task and Meta Learning; University of Michigan EECS 498-007: Deep Learning for Computer Vision; OpenCV Course: Full Tutorial with Python (freeCodeCamp)

Additional Information

Languages: English (Professional; IELTS Academic overall band 6.5, Aug 2026), Urdu (Native)
Interests: Few-Shot and Zero-Shot Generative Learning, Vision-Language and Multimodal Learning, Deep Generative Models, Computer Vision, Explainable AI, Language Models and Program Comprehension

Achievements

Key Achievements:

- Received a **merit-based scholarship** from NED University of Engineering and Technology for three semesters of the MS in Data Science, and graduated with a CGPA of **3.81/4.00**.
- Ranked among the **top 20 Computer Science students in Karachi** in intermediate (Sir Syed Government Girls College) and received a **cash prize from the Prime Minister of Pakistan**.
- **Presented a research paper** at the IEEE conference FET'26 (University of Bahrain, online), 22–23 Apr 2026.
- Presented my BS final year project at the codeBIT FYP Symposium, University of Karachi, Mar 2022.